

0.55.2. La vitesse des déplacements : vitesse — doublé, deux positions

Un seul `vitesse(100)` pour les deux : il vaut pour tous les déplacements qui suivent.

1. Ce qu'il faut comprendre

C'est le 0.55 et le 0.55.1 réunis : le A puis le B, avec le même `vitesse(100)`.

Un seul réglage suffit : `vitesse` reste réglée jusqu'au suivant, pour toutes les lettres.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1619 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 0;      // le A
```

2. UNE VARIABLE — XB

```
uint8_t xb = 0;     // le B
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    vitesse(100);           // pour les deux
    deplace_x(x, 0, ALPHABET[0], 5);
    deplace_x(xb, 8, ALPHABET[1], 5);

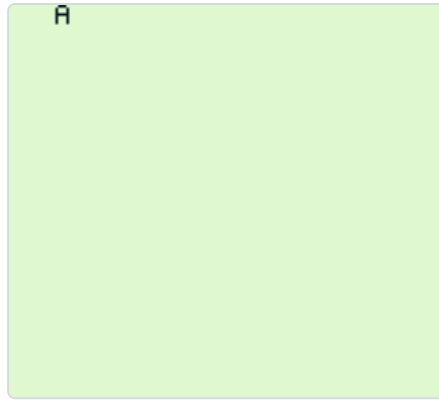
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

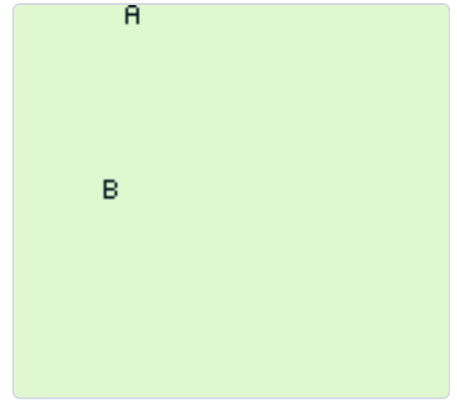
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



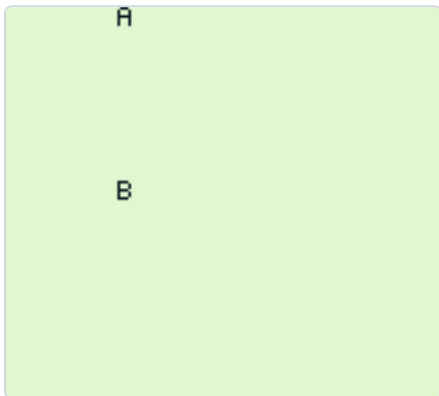
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le A file sur la ligne 0, puis le B sur la ligne 8, à la même vitesse.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (5, 0) (5,0)

✓ le B finit en (5, 8) (5,8)